

C++ Homework Foundation

Loops & Conditionals Course Assignment Guide

Lab Session Instructor Resource

Compiled on: July 6, 2026

Abstract

This guide contains the official answer key, syntax explanations, and structural templates for the 10 fundamental C++ programming homework tasks. Designed specifically for absolute beginners, these examples prioritize clarity, simple console I/O, and clear logical branches.

Contents

1	Part 1: Pure Conditionals (if-else)	2
1.1	Problem 1: The Even-Odd Splitter	2
1.2	Problem 2: Theater Ticket Pricing	2
1.3	Problem 3: Positive, Negative, or Zero?	4
1.4	Problem 4: The Leap Year Secret (Simplified)	4
2	Part 2: Pure Loops (for)	5
2.1	Problem 5: Multiplication Table of 5	5
2.2	Problem 6: Summing the First N Numbers	5
2.3	Problem 7: The Step-Counter Loop	6
3	Part 3: Combining Loops & Conditionals	6
3.1	Problem 8: Multiples of 3 Filter	6
3.2	Problem 9: Grade Statistics Tracker	7
3.3	Problem 10: The FizzBuzz Intro	7

1 Part 1: Pure Conditionals (if-else)

Focus: Mastering logical comparisons, syntax structures, and checking range boundaries.

1.1 Problem 1: The Even-Odd Splitter

Concept Explanation

The **% (modulo)** operator finds the integer remainder of a division. When dividing any integer by 2, a remainder of 0 indicates an even number, while a remainder of 1 indicates an odd number.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int num;
6     cout << "Enter a single integer: ";
7     cin >> num;
8
9     if (num % 2 == 0) {
10         cout << num << " is an Even number." << endl;
11     } else {
12         cout << num << " is an Odd number." << endl;
13     }
14
15     return 0;
16 }
```

1.2 Problem 2: Theater Ticket Pricing

Concept Explanation

This problem tests sequential evaluations. The conditions are checked from top to bottom. The first condition that evaluates to `true` runs its respective block, and the rest are entirely skipped.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int age;
6     cout << "Enter your age: ";
7     cin >> age;
8
9     if (age < 12) {
10         cout << "The ticket price is $5." << endl;
11     } else if (age >= 60) {
12         cout << "The ticket price is $7." << endl;
13     } else {
14         cout << "The ticket price is $10." << endl;
15     }
16 }
```

```
17     return 0;  
18 }
```

1.3 Problem 3: Positive, Negative, or Zero?

Concept Explanation

This program covers simple comparison operators (>, <, and ==). It is a great opportunity to remind students that a single = is used to assign values, while a double == is used to check for equality.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int num;
6     cout << "Enter a number: ";
7     cin >> num;
8
9     if (num > 0) {
10         cout << "Positive" << endl;
11     } else if (num < 0) {
12         cout << "Negative" << endl;
13     } else {
14         cout << "You entered Zero" << endl;
15     }
16
17     return 0;
18 }
```

1.4 Problem 4: The Leap Year Secret (Simplified)

Concept Explanation

By checking if a year is perfectly divisible by 4, we teach students how to model standard repeating schedules. In C++, evaluating `year % 4 == 0` confirms this division.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int year;
6     cout << "Enter a year: ";
7     cin >> year;
8
9     if (year % 4 == 0) {
10         cout << "It is a Leap Year!" << endl;
11     } else {
12         cout << "It is a regular year." << endl;
13     }
14
15     return 0;
16 }
```

2 Part 2: Pure Loops (for)

Focus: Tracking initialization, defining conditions, and modifying loop counters dynamically.

2.1 Problem 5: Multiplication Table of 5

Concept Explanation

The loop control variable `i` acts as our multiplier. It starts at 1 and ticks upward sequentially until it reaches 10, generating the math calculations dynamically on each run.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     for (int i = 1; i <= 10; i++) {
6         cout << "5 x " << i << " = " << 5 * i << endl;
7     }
8     return 0;
9 }
```

2.2 Problem 6: Summing the First N Numbers

Concept Explanation

This introduces an **accumulator pattern**. Students must declare a running variable (like `sum`) *outside* the loop and initialize it to 0 before adding values to it across the iterations.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n;
6     int sum = 0; // Crucial: Initialize to 0!
7
8     cout << "Enter a positive integer N: ";
9     cin >> n;
10
11     for (int i = 1; i <= n; i++) {
12         sum = sum + i;
13     }
14
15     cout << "The total sum is " << sum << endl;
16     return 0;
17 }
```

2.3 Problem 7: The Step-Counter Loop

Concept Explanation

This breaks the common habit of always writing `i++`. Students learn they can change how quickly a loop climbs by using expression patterns like `i += 5` (equivalent to `i = i + 5`).

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // Start at 5, loop while i is <= 50, jump by 5 each time
6     for (int i = 5; i <= 50; i += 5) {
7         cout << i << " ";
8     }
9     cout << endl; // Line break for terminal cleanliness
10    return 0;
11 }
```

3 Part 3: Combining Loops & Conditionals

Focus: Nesting checks inside repeat cycles to filter data sets dynamically.

3.1 Problem 8: Multiples of 3 Filter

Concept Explanation

The loop steps systematically through every index between 1 and 30, but the inner `if` statement acts as a gatekeeper, printing only values that meet our divisibility requirements.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     for (int i = 1; i <= 30; i++) {
6         if (i % 3 == 0) {
7             cout << i << " ";
8         }
9     }
10    cout << endl;
11    return 0;
12 }
```

3.2 Problem 9: Grade Statistics Tracker

Concept Explanation

Here, the loop works to collect multiple inputs from the console. By combining input statements and conditional counters, we keep score of valid records across our runs.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int grade;
6     int brilliantStudents = 0; // State variable outside loop
7
8     for (int i = 1; i <= 5; i++) {
9         cout << "Enter grade for student " << i << ": ";
10        cin >> grade;
11
12        if (grade >= 80) {
13            brilliantStudents++; // Increment positive counters
14        }
15    }
16
17    cout << "Number of brilliant students: " << brilliantStudents << endl;
18    return 0;
19 }
```

3.3 Problem 10: The FizzBuzz Intro

Concept Explanation

This mimics the industry-standard "FizzBuzz" structural exercise. Students route control flows inside a cycle to alternate between terminal text streams and numeric displays.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     for (int i = 1; i <= 15; i++) {
6         if (i % 5 == 0) {
7             cout << "Buzz ";
8         } else {
9             cout << i << " ";
10        }
11    }
12    cout << endl;
13    return 0;
14 }
```